

DMD Controller — Manuale di installazione

Servizio che pilota il pannello LED 256×64 (chip FM6373) da Raspberry Pi, si presenta a Batocera come un **ZeDMD-WiFi** e offre un'interfaccia web di controllo.

Pacchetto: `dmdcontroller.tar.gz`

1. Prerequisiti

Il Raspberry deve essere già preparato secondo la procedura hardware del progetto:

- audio integrato disattivato (`dtparam=audio=off` in `/boot/firmware/config.txt`)
- `isolcpus=3` in `/boot/firmware/cmdline.txt` sui modelli quad-core
- libreria `rpi-rgb-led-matrix_pwm_experiment` clonata e compilata nella home
- pannelli cablati e già verificati con la demo del quadrato rotante

Serve inoltre l'accesso SSH al Raspberry (in questo manuale: utente `gillo` , hostname `dmdpi`).

2. Trasferimento dei file

Dal **Mac**, nella cartella dove hai scaricato il pacchetto:

```
cd ~/Downloads
tar xzf dmd*controller*.tar.gz
scp -r dmd gillo@dmdpi.local:~/
```

Il carattere jolly `dmd*controller*` copre le varianti del nome file introdotte dal browser in fase di download.

3. Installazione

Via SSH sul Raspberry:

```
ssh gillo@dmdpi.local
cd ~/dmd
chmod +x install.sh
sudo ./install.sh
```

Lo script esegue in sequenza:

1. installazione dei pacchetti di sistema (Flask, NumPy, Pillow, Cython, font)
2. **compilazione dei binding Python** della libreria matrice
3. copia dei file in `/opt/dmd`

4. creazione della configurazione in `/etc/dmd/config.json`

5. registrazione del servizio systemd `dmd`

Il passo 2 è lento. Compila da sorgente l'intera libreria C++: da qualche minuto su Pi 4 fino a oltre dieci minuti su Pi Zero 2 W. È normale, non interrompere. Il cursore che gira indica che sta lavorando.

Se la libreria matrice non si trova nella home dell'utente:

```
sudo MATRIX_DIR=/percorso/della/libreria ./install.sh
```

4. Configurazione prima del primo avvio

Apri `/etc/dmd/config.json` e verifica due valori:

Chiave	Valore
<code>panel slowdown</code>	1 Pi Zero W · 3 Pi Zero 2 W e Pi 3 · 5 Pi 4
<code>panel chain</code>	numero di pannelli collegati in cascata

Le altre chiavi rilevanti, già corrette di default:

Chiave	Valore	Significato
<code>web.port</code>	8080	interfaccia web
<code>zedmd.http_port</code>	80	handshake ZeDMD, non modificare
<code>zedmd.stream_port</code>	3333	frame DMD (TCP e UDP)
<code>zedmd.grace_seconds</code>	60	quanto ZeDMD trattiene il display dopo l'ultimo frame
<code>zedmd.client_timeout</code>	10	silenzio oltre il quale il client è considerato caduto

5. Avvio

```
sudo systemctl start dmd
journalctl -u dmd -f
```

Nel log devono comparire le righe del server handshake sulla porta 80, del ricevitore sulla 3333 e dell'interfaccia web sulla 8080. Si esce dal log con `Ctrl+C`: il servizio continua a funzionare.

Il servizio è già abilitato all'avvio automatico. Per verificarlo, riavvia una volta il Raspberry e controlla che riparta da solo.

Interfaccia web: `http://dmdpi.local:8080/` Digitando l'indirizzo senza porta si viene rediretti automaticamente.

6. Le tre porte

Porta	Uso	Servita da
80	handshake ZeDMD (cablata nel client, non modificabile)	server HTTP dedicato
3333	frame DMD, TCP e UDP	ricevitore ZeDMD
8080	interfaccia web	Flask

L'handshake **non** passa da Flask di proposito: il client di libzedmd legge la risposta con una sola `recv()` e si ferma appena riceve meno di 1024 byte. Se header e corpo arrivano in due pacchetti distinti — come fa Flask — il client legge un corpo vuoto, non riconosce il trasporto TCP e ripiega su UDP. Il server dedicato invia tutto in una sola scrittura.

7. Collegamento a Batocera

Sulla macchina Batocera, via SSH come `root`, crea il file di configurazione di `dmdserver`:

```
cat > /userdata/system/configs/dmdserver/config.ini << 'EOF'
[DMDServer]
AltColor = 1

[ZeDMD]
Enabled = 0

[ZeDMD-WiFi]
Enabled = 1
WiFiAddr = 192.168.0.XXX
EOF
```

Sostituisci `192.168.0.XXX` con l'indirizzo IP del Raspberry, che trovi nella pagina **Impostazioni** dell'interfaccia web.

`[ZeDMD] Enabled = 0` disattiva la ricerca del dispositivo su porta seriale: con il collegamento di rete attivo non serve ed eviterebbe conflitti.

Poi, nel menu di Batocera, attiva il servizio **DMD reale** (non "DMD Web", che è il simulatore su browser).

Verifica

Con `journalctl -u dmd -f` aperto sul Raspberry, riavvia il servizio DMD di Batocera. Deve comparire:

```
[zedmd] client connesso via TCP: ('192.168.0.XXX', ...)
```

Se invece vedi ripetutamente solo `GET /handshake` senza connessione, il client non sta riuscendo ad aprire lo stream: verifica che `zedmd.http_port` sia 80 e `web.port` sia 8080.

8. Aggiornamento

Quando ricevi una nuova versione del pacchetto, dal Mac:

```
cd ~/Downloads
tar xzf dmd*controller*.tar.gz
scp -r dmd gillo@dmdpi.local:~/
```

Poi sul Raspberry:

```
cd ~/dmd && sudo ./update.sh
```

Lo script copia i file aggiornati in `/opt/dmd`, adegua automaticamente la configurazione esistente senza sovrascrivere le tue impostazioni, e riavvia il servizio.

Riavvia sempre Batocera dopo un aggiornamento. Il client ZeDMD mantiene in memoria lo stato della connessione e la contabilità interna delle zone dell'immagine già inviate. Dopo il riavvio del servizio sul Raspberry quello stato non è più valido: senza un riavvio di Batocera il display può restare fermo sull'ultimo contenuto o aggiornarsi solo parzialmente.

L'aggiornamento **non** richiede di ricompilare i binding Python: quel passo si esegue una volta sola in fase di installazione.

9. Interfaccia web

Impostazioni — luminosità con applicazione immediata sul pannello, server NTP, fuso orario, ora legale automatica o scostamento UTC manuale, indirizzo IP locale, stato della sincronizzazione oraria e riepilogo delle porte.

Servizi — attivazione dei quattro servizi, indicazione della sorgente attualmente a schermo e possibilità di forzarne una invece di lasciar decidere l'arbitro:

Servizio	Stato
ZeDMD	funzionante
Media Player - Clock	orologio funzionante, slideshow da implementare
Status Player	da implementare
Air Radar	da implementare

10. Come viene deciso cosa appare sul display

Un solo processo può pilotare i GPIO, quindi tutte le sorgenti vivono nello stesso servizio e un arbitro sceglie chi vince:

Priorità	Sorgente
100	ZeDMD
10	Media Player - Clock

ZeDMD prende il controllo **immediatamente** appena un client si connette o arriva un frame, e lo mantiene per `grace_seconds` dopo l'ultimo segnale. Il tempo di grazia serve a evitare che l'orologio si intrometta durante le pause normali di Batocera: menu, caricamenti, cambi di schermata.

Se Batocera viene spento bruscamente, la connessione non viene chiusa in modo pulito: il ricevitore se ne accorge dopo `client_timeout` secondi di silenzio, poi decorre il tempo di grazia. Con i valori di default il display torna all'orologio entro circa 70 secondi.

11. Comandi utili

```
sudo systemctl start dmd      # avvia
sudo systemctl stop dmd      # ferma
sudo systemctl restart dmd    # riavvia
systemctl status dmd         # stato
journalctl -u dmd -f          # log in tempo reale
journalctl -u dmd -n 100      # ultime 100 righe
curl http://localhost/handshake # verifica dell'handshake
```

12. Risoluzione problemi

Sintomo	Causa probabile	Rimedio
Il servizio non parte, errore su <code>rgbmatrix</code>	binding Python non compilati	rilanciare <code>install.sh</code>
Pannello nero ma servizio attivo	nessuna sorgente abilitata	attivare Media Player - Clock dalla pagina Servizi
<code>Address already in use</code> sulla porta 80	un altro web server è attivo	fermarlo (<code>lighttpd</code> , <code>apache2</code> , <code>nginx</code>)
Handshake ripetuto, nessun frame	porte scambiate	<code>zedmd.http_port = 80</code> , <code>web.port = 8080</code>
Immagine a scatti o righe sporadiche	<code>panel slowdown</code> errato per il modello	correggere e riavviare il servizio
L'orologio interrompe Batocera	tempo di grazia troppo corto	alzare <code>zedmd.grace_seconds</code>
Dopo un aggiornamento il display resta fermo	stato del client non più valido	riavviare Batocera
Selezioni rapide: il display resta indietro di un passo	il client scarta i frame intermedi	verificare che il contatore frame avanzi nella pagina Servizi

13. Struttura dei file installati

```
/opt/dmd/dmdd.py      servizio principale, arbitro e ciclo di rendering
/opt/dmd/display.py   proprietario esclusivo del pannello
/opt/dmd/dmdconf.py   configurazione persistente
/opt/dmd/webui.py      interfaccia web (Flask)
/opt/dmd/zedmd_http.py server dell'handshake ZeDMD sulla porta 80
/opt/dmd/sources/      sorgenti di contenuto
/etc/dmd/config.json   configurazione
/etc/systemd/system/dmd.service
```